20 Agentic Design Patterns (Part 1)

A comprehensive guide to 20 agentic design patterns that separate pros from beginners, based on a Google engineer's 400-page book. Practical patterns you can use today with plain English explanations.

Back

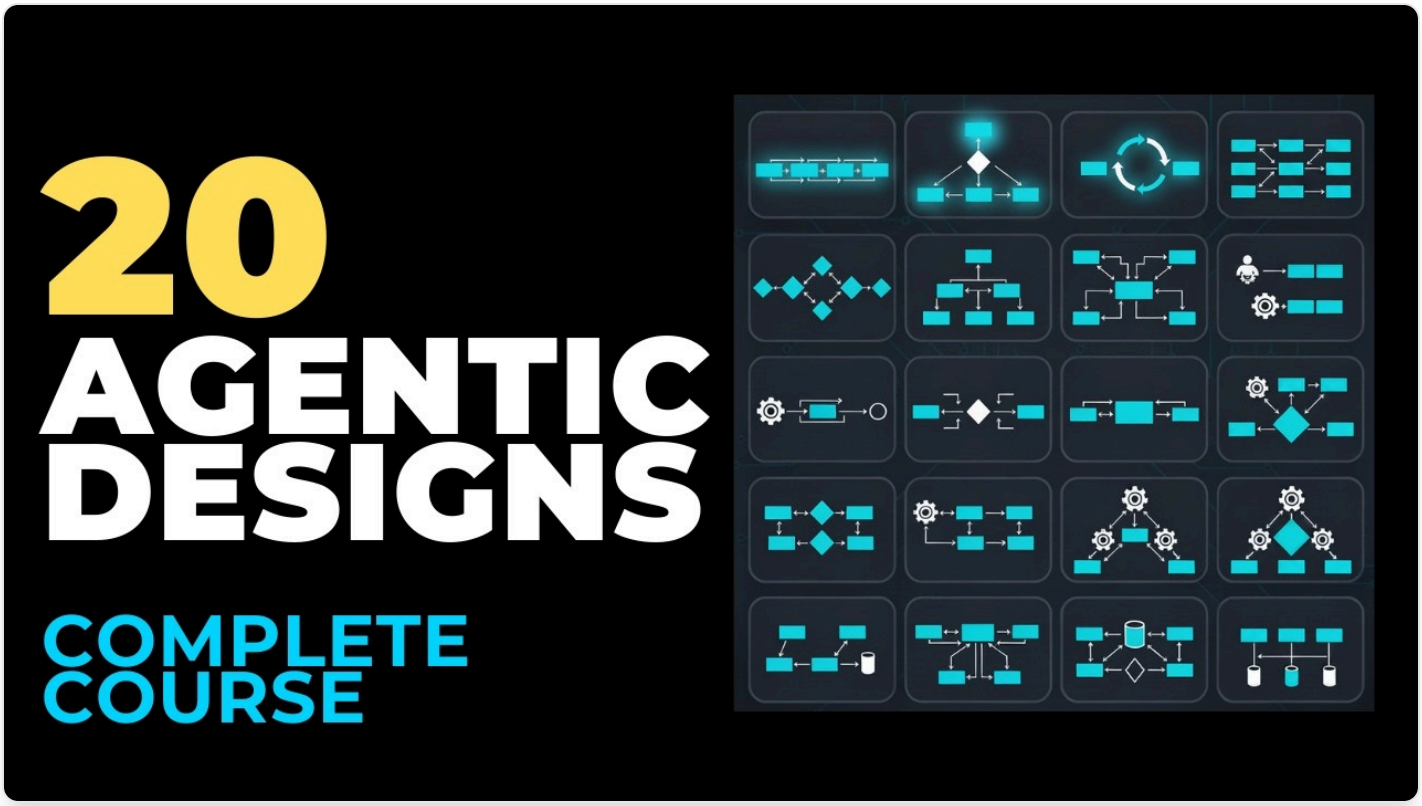
This guide is split into 4 parts for better performance:

- Part 1: Chapters 1-5 - Prompt Chaining, Routing, Parallelization, Reflection, Tool Use
- Part 2: Chapters 6-10 - Planning, Multi-Agent Collaboration, Memory Management, Learning and Adaptation, Goal Setting and Monitoring
- Part 3: Chapters 11-15 - Exception Handling and Recovery, Human in the Loop, Knowledge Retrieval (RAG), Inter-Agent Communication, Resource-Aware Optimization
- Part 4: Chapters 16-20 - Reasoning Techniques, Evaluation and Monitoring, Guardrails and Safety Patterns, Prioritization, Exploration and Discovery

Introduction

You can help out the author who broke down the 400-page manual published by the Google engineer here.

Link not affiliated.



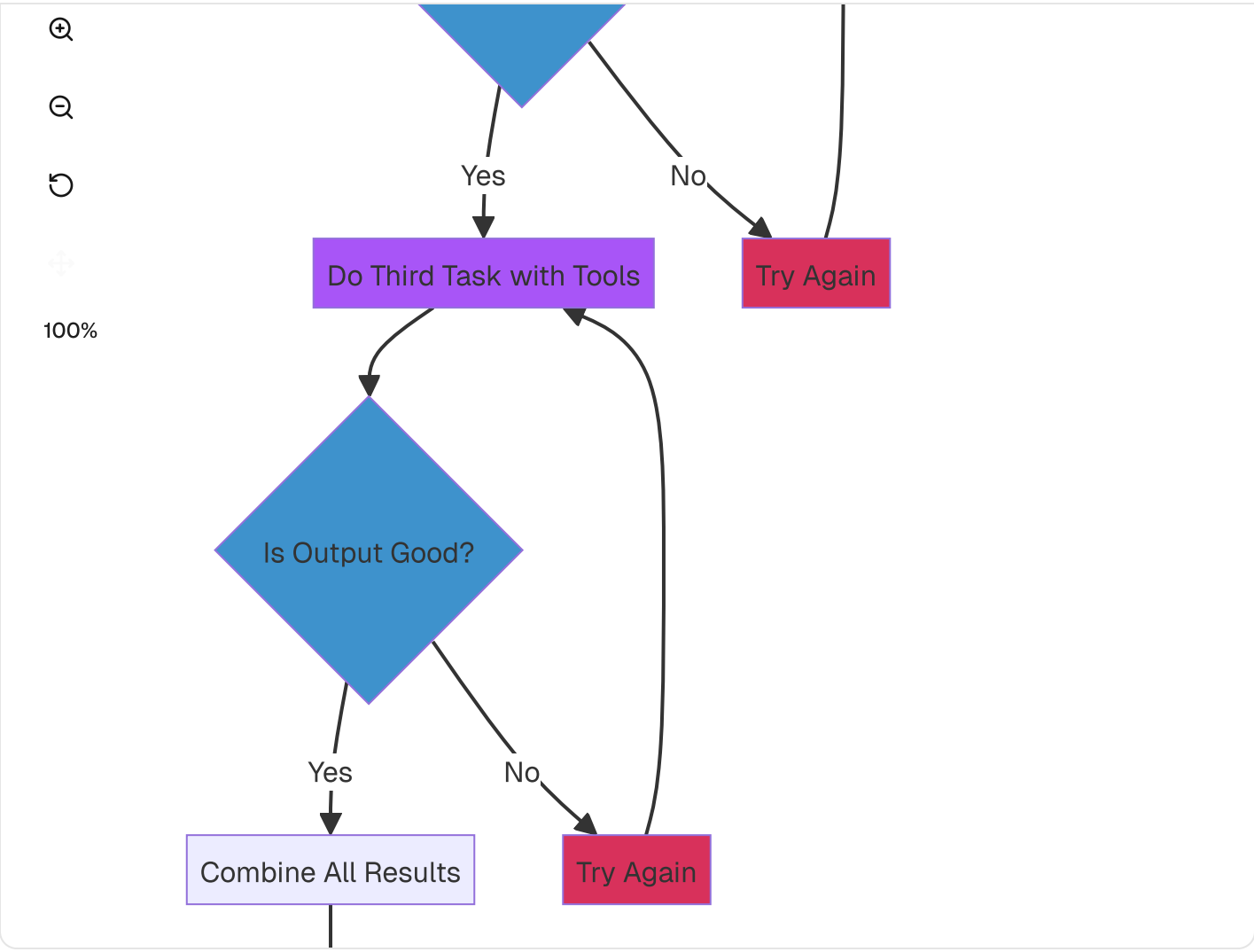
Chapter 1: Prompt Chaining

TLDDR: Breaking big tasks into smaller sequential steps, where each step validates the output of the previous one before passing data to the next. Think of it like an assembly line where each station completes its part, checks quality, then hands it off to the next section.

When to Use Where It Fits

- **Complex multi-step processes**: When you need to break down a complex task into discrete, manageable steps
- **Data transformation pipelines**: When information needs to be processed through multiple stages with different requirements
- **Quality-critical workflows**: When each step's output must meet specific criteria before proceeding
- **Debugging requirements**: When you need clear visibility into each stage of processing
- **Mixed tool/AI operations**: When combining LLM calls with API calls, database queries, or other tools

How It Works



How does prompt chaining work?

Break big task into subtasks, create data contracts between them

Data contracts?

Rules for what each step outputs and what the next step expects

Then what?

How many steps?

Usually 3-5. Too many and you get diminishing returns or hallucinations

What if something fails?

Log everything. You can trace back through the chain to see exactly where it broke

Pros Cons

Pros

- **Modularity**: Each step can be developed, tested, and optimized independently
- **Debuggability**: Clear visibility into where failures occur in the chain
- **Reliability**: Structured data contracts ensure consistent handoffs between steps
- **Reusability**: Individual chain components can be reused in different workflows
- **Error handling**: Each step can have specific retry logic and fallback strategies
- **Incremental progress**: Partial results can be saved and resumed if interrupted
- **Parallel development**: Different team members can work on different chain segments

Real-World Examples

Legal Documents E-commerce Research Bug Analysis Financial Reports

Legal Document Analysis

- Step 1: Extract key clauses from contracts

- Step 4: Generate executive summary with recommendations

Chapter 2: Routing

TLDDR: Analyzing incoming requests and sending them to the right specialist agent based on what they need. Think of it like a smart receptionist or operator who listens to what you need and directs you to the right person or department—tech support, accounting, etc. The key thing is if the operator is unsure, they should go back and ask clarifying questions to better understand where to route that request.

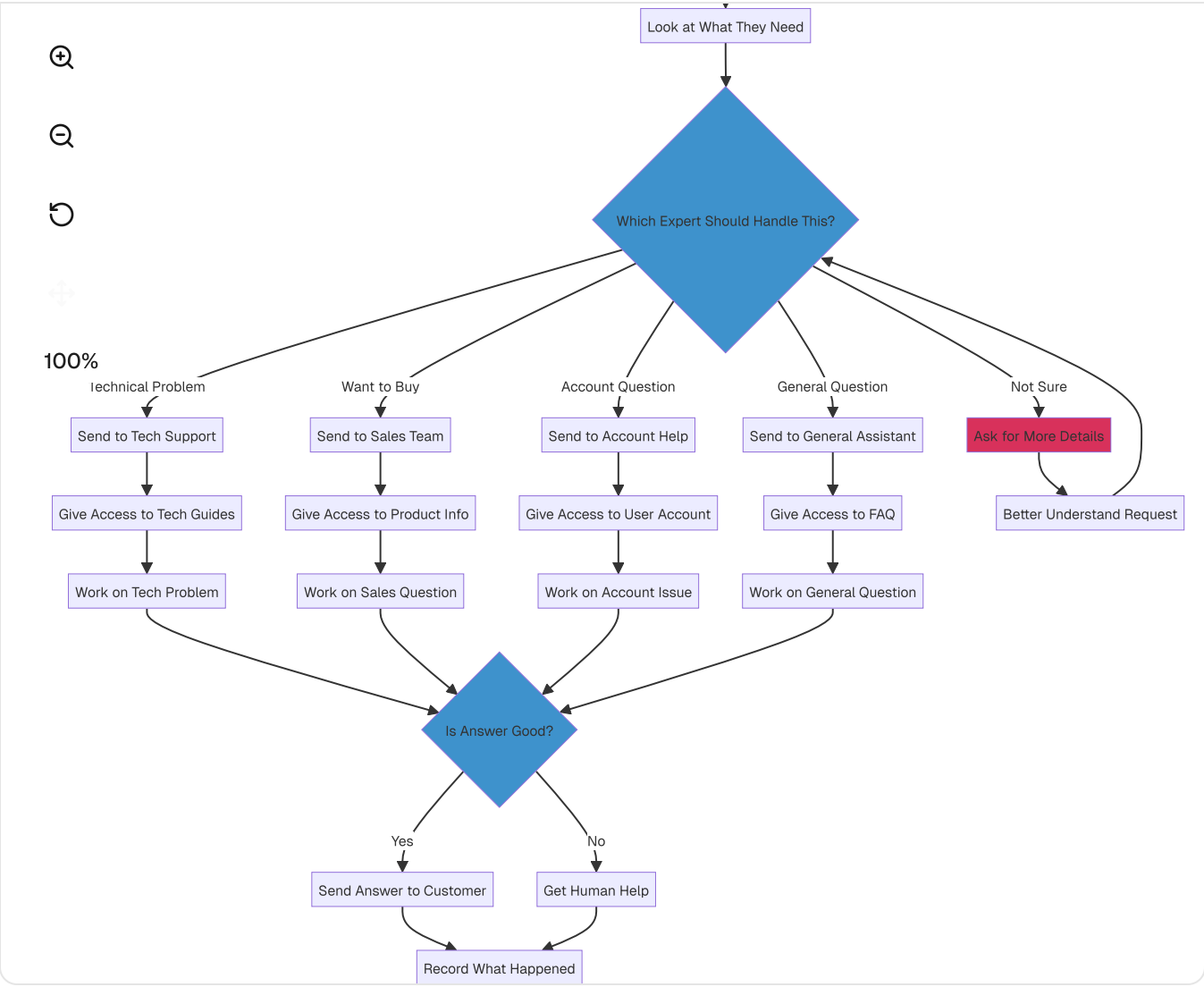
When to Use Where It Fits

When to Use

- **Multi-domain systems**: When handling diverse request types requiring different expertise
- **Dynamic workflow selection**: When the appropriate process depends on input characteristics
- **Resource optimization**: When different requests require different computational resources
- **Specialized tool access**: When specific tools or APIs are needed based on request type
- **Confidence-based processing**: When you need to handle ambiguous requests differently
- **Load balancing**: When distributing work across multiple specialized agents

How It Works

MermaidZoom MMD



How does routing work?

Analyze the request, decide which specialist agent should handle it

What if it's not sure?

Ask clarifying questions until confidence is high enough

How do you measure confidence?

LLM can give a score out of 10, but that's unreliable. Better to use deterministic stats

Then what?

Route to the right agent, get response, check if it succeeded or failed, deliver result

Pros Cons

Pros

- **Specialization**: Each route can be optimized for specific task types
- **Scalability**: Easy to add new routes without affecting existing ones
- **Efficiency**: Requests are handled by the most appropriate resources
- **Flexibility**: Dynamic routing based on context and confidence
- **Clarity**: Clear separation of concerns between different workflows
- **Performance**: Avoid unnecessary processing for simple requests
- **Maintainability**: Each route can be updated independently

Real-World Examples

Customer Service Content Creation Code Assistant Financial Services Education

AI Customer Service Hub

- Technical issues → Technical Support Agent with access to documentation
- Billing questions → Finance Agent with access to payment systems
- Product inquiries → Sales Agent with catalog access
- Complaints → Escalation Agent with CRM integration

Chapter 3: Parallelization

TLDDR: Splitting a large job into independent chunks that can be processed at the same time by multiple workers. When we say workers here, that is a proxy for agents. Think of it like having 10 people each read different chapters of a book simultaneously, then combining all the summaries at the end.

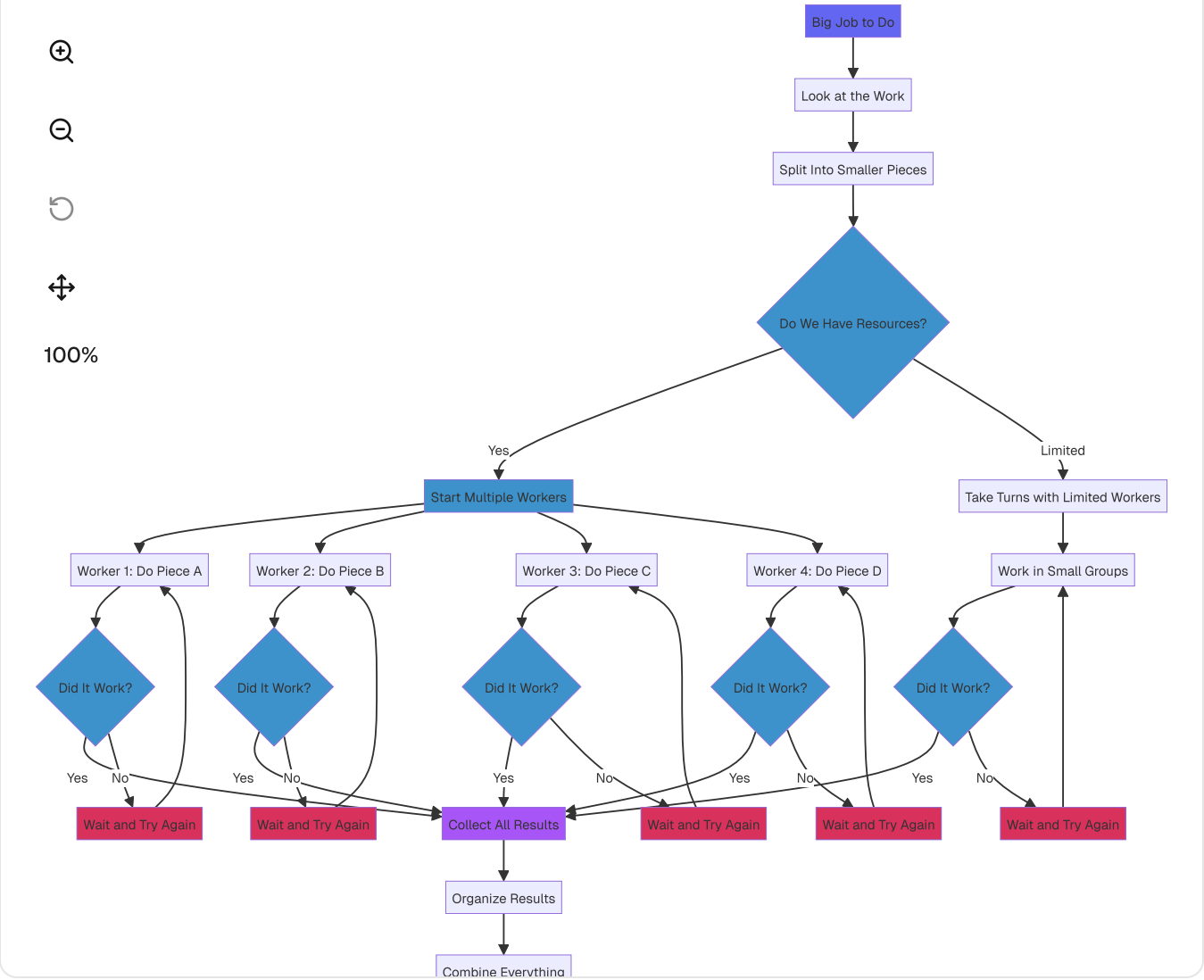
When to Use Where It Fits

When to Use

- **Large-scale data processing**: When processing multiple documents, records, or data sources
- **Time-sensitive operations**: When results are needed quickly and tasks are independent
- **Batch operations**: When performing the same operation on multiple items
- **Web scraping/crawling**: When gathering data from multiple sources simultaneously
- **Multi-document analysis**: When analyzing multiple files or documents independently
- **API aggregation**: When calling multiple APIs that don't depend on each other

How It Works

MermaidZoom MMD



How does parallelization work?

Take big task, split into independent chunks that can run at the same time

Example?

Like reducing customer churn by 20%. Split into: survey customers, analyze exit interviews, check platform issues

Check resources, spawn parallel workers. Each worker retries until it succeeds

How do you combine results?

Normalize everything into same format (turn apples/oranges into all apples), merge, create summary

Why track which worker did what?

So if something fails, you know which worker to fix or adjust

Pros Cons

Pros

- **Speed improvement**: Dramatic reduction in total processing time
- **Resource utilization**: Better use of available computational resources
- **Scalability**: Easy to scale up or down based on workload
- **Fault isolation**: Failure in one worker doesn't affect others
- **Progress tracking**: Can show incremental progress as workers complete
- **Flexibility**: Can dynamically adjust worker count based on load
- **Cost efficiency**: Optimize resource usage and reduce idle time

Real-World Examples

News Aggregation Price Monitoring Document Intelligence Social Media Code Analysis Translation

- Simultaneously fetch articles from 50+ news sources
- Each worker processes one news source
- Rate limit to 10 concurrent API calls
- Merge and deduplicate results
- Sort by relevance and timestamp

Chapter 4: Reflection

TLDDR: Generate a first draft, then have a critic review it against quality standards. Based on the feedback, revise and improve. Essentially repeat this until you meet your quality standards. Think of it like writing an essay, having a teacher review it for you, and then making improvements until you finally get a passing grade.

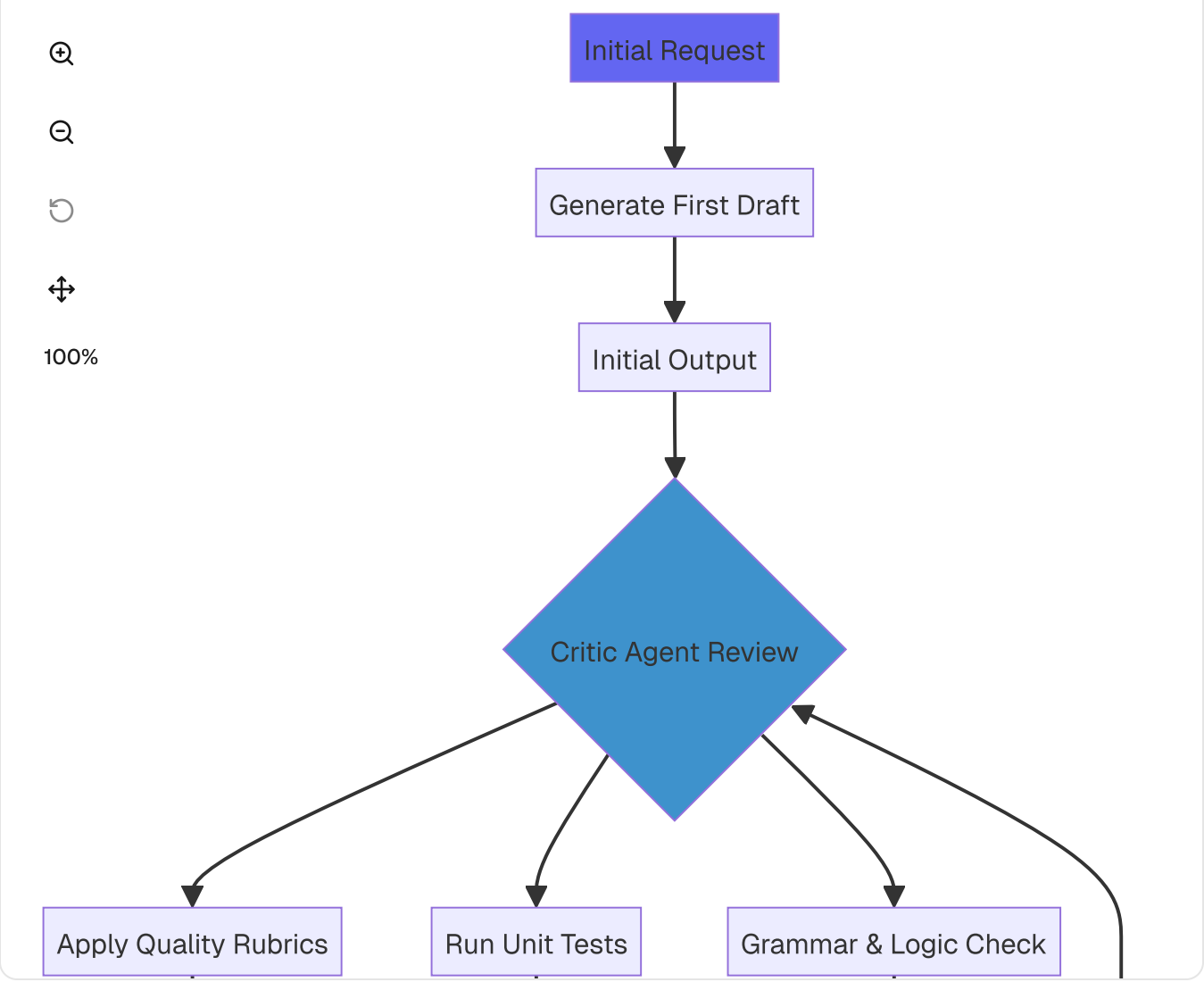
When to Use Where It Fits

When to Use

- **Quality-critical outputs**: When high accuracy and quality are non-negotiable
- **Complex reasoning tasks**: When problems require iterative refinement
- **Creative work**: When content needs multiple rounds of improvement
- **Learning systems**: When you want to improve performance over time
- **Error-prone domains**: When initial attempts often have mistakes
- **Compliance requirements**: When outputs must meet specific standards

How It Works

MermaidZoom MMD



How does reflection work?

Generate first draft, then critic agent reviews it against quality standards

What standards?

Quality rubrics, unit tests for edge cases, grammar/logic checks

If it passes?

Accept output, record success patterns, update prompts if needed

If it fails?

Generate structured feedback, send back to original agent, revise and repeat

What if it keeps failing?

Set max iterations (like 3 tries). After that, use the best version you got

Pros Cons

Pros

- **Quality improvement**: Systematic enhancement through multiple iterations
- **Error reduction**: Catches and fixes mistakes before final delivery
- **Objectivity**: Separation of generation and critique roles
- **Learning capability**: System improves over time from patterns
- **Transparency**: Clear feedback trail for improvements
- **Flexibility**: Can adjust critique criteria for different use cases
- **Consistency**: Applies same quality standards uniformly

Real-World Examples

Blog Post Contract Generation Education Documentation Marketing Research

Technical Blog Post Creation

- Initial draft generation

- SEO optimization check
- Readability improvements
- Final grammar and style polish

Chapter 5: Tool Use

TLDDR: When the AI needs external information or actions, it discovers available tools, checks permissions, and then calls the right tool with proper parameters. Think of it like a chef who needs ingredients, checks what's available in the pantry, then verifies they can use it, and then retrieves and actually uses it in the recipe.

When to Use Where It Fits

When to Use

- **External data access**: When agents need real-time or dynamic information
- **System integration**: When connecting to databases, APIs, or services
- **Computational tasks**: When precise calculations or data processing is needed
- **File operations**: When reading, writing, or manipulating files
- **Action execution**: When agents need to perform concrete actions
- **Multi-step workflows**: When combining AI reasoning with tool execution

How It Works

MermaidZoom MMD

```
graph TD
    Start[User Request] ⟶ Analyze[Analyze Task Requirements]
    Analyze ⟶ Discover[Discover Available Tools]
```

```
    Catalog ⟶ API2[Database Query Tool]
    Catalog ⟶ API3[Calculator Function]
    Catalog ⟶ API4[File System Access]
    Catalog ⟶ API5[External Service API]

    Catalog ⟶ Select{Select Appropriate Tool}

    Select ⟶ Match[Match Capabilities to Need]
    Match ⟶ Safety{Safety Check}

    Safety ⟶|Pass| Prepare[Prepare Tool Call]
    Safety ⟶|Fail| Deny[Deny Access with Reason]

    Prepare ⟶ Validate[Validate Input Parameters]
    Validate ⟶ Call[Execute Tool with Arguments]

    Call ⟶ Handle{Handle Response}
    Handle ⟶|Success| Parse[Parse Tool Output]
    Handle ⟶|Error| ErrorHandle[Error Recovery]
    Handle ⟶|Timeout| Retry[Retry with Backoff]

    ErrorHandle ⟶ Fallback[Use Fallback Method]
    Retry ⟶ Call

    Parse ⟶ Normalize[Normalize for LLM]
    Fallback ⟶ Normalize
```

How does tool use work?

Analyze request, discover available tools, select the right one

What tools?

Web search API, database queries, calculator, file system, other APIs

Then what?

Safety check - is this the right tool? If yes, prepare and execute tool call

What if it fails?

Retry with backoff, or use fallback method. Normalize output for LLM to understand

Built by neldivad. Copyright © 2024 - 2026 Nelworks Inc.

What if wrong tool was chosen?

Deny access, log the reason, so you can fix the workflow later

Pros Cons

Pros

- **Capability extension**: Agents can perform actions beyond text generation
- **Real-time data**: Access to current information not in training data
- **Precision**: Exact calculations and deterministic operations
- **Integration**: Seamless connection to existing systems and services
- **Automation**: Complete end-to-end workflows without human intervention
- **Flexibility**: Dynamic tool selection based on task requirements
- **Auditability**: Clear log of all tool usage and parameters

Real-World Examples

Financial Analysis Code Development E-commerce Research Paper Smart Home HR Recruitment

Code Development Helper

- File system access for reading/writing code
- Compiler/interpreter for code execution

- Documentation generators